Introduction to OpenMP

Lecture 6: Further topics in OpenMP

Overview

- Nested parallelism
- Orphaned constructs
- Thread-private globals
- User defined reductions
- Construct cancellation
- Timing routines

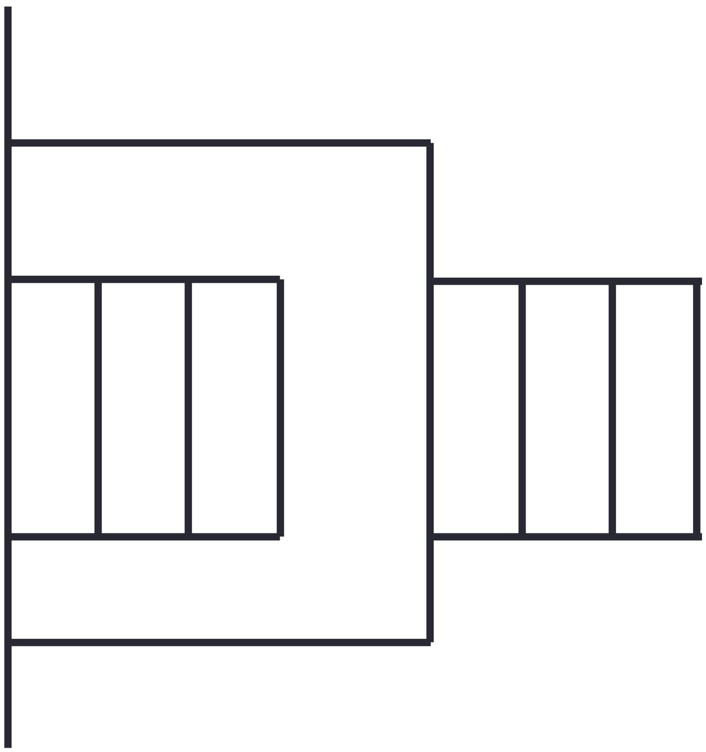
Nested parallelism

- Nested parallelism is supported in OpenMP.
  - If a `parallel` construct is encountered within another `parallel` construct, another new team of threads will be created.
  - Not often needed, but can be useful if the outer level does not contain enough parallelism
- Usually disabled by default
  - number of active levels of parallelism is set to 1 by default
  - can increase it with the `OMP_MAX_ACTIVE_LEVELS` environment variable or the `omp_set_max_active_levels()` routine.
  - some performance penalty: don't enable nested parallelism unless you are using it!
- If the number of active levels of parallelism is exceeded, code with further nested parallel regions will still execute, but the inner teams will contain only one thread.

Nested parallelism (cont)

Example:

```
#pragma omp parallel
{
int myid = omp_get_thread_num();
if (myid == 0) {
#pragma omp parallel for
      for(int i=0; i<N; i++){
          x[i] = 1.0;
      }
}
else {
#pragma omp parallel for
      for(int j=0; i<M; i++){
          y[i] = 1.0;
      }
}
}
```



Controlling the number of threads

- Can use the environment variable

```
export OMP_NUM_THREADS=2,4
```

- Will use 2 threads at the outer level and 4 threads for each of the inner teams.

- Can use `omp_set_num_threads()` or the `num_threads` clause on the parallel region.
- Can also control the maximum number of threads running at any one time.

```
export OMP_THREAD_LIMIT=64
```

omp_set_num_threads()

- Useful if you want inner regions to use different numbers of threads:

```
omp_set_num_threads(2);
#pragma omp parallel for
  for (int i=0; i<4; i++) {
    omp_set_num_threads(innerthreads[i]);
#pragma omp parallel for
     for (int j=0; j<N; j++) {
         a[j][i] = b[j][i] * 17;
     }
  }
```

- The value set overrides the value(s) in the environment variable **OMP_NUM_THREADS**

`num_threads` clause

- Another way to control the number of threads used at each level is with the `num_threads` clause:

```
#pragma omp parallel for num_threads(2)
  for (int i=0; i<4; i++) {
#pragma omp parallel for num_threads(innerthreads[i])
     for (int j=0; j<N; j++) {
         a[j][i] = b[j][i] * 17;
     }
  }
```

- The value set in the clause overrides the value in the environment variable `OMP_NUM_THREADS` and that set by `omp_set_num_threads()`

Utility routines for nested parallelism

- **`omp_get_level()`**
  - returns the level of parallelism of the calling thread
  - returns 0 in the sequential part
- **`omp_get_active_level()`**
  - returns the level of parallelism of the calling thread, ignoring levels which are inactive (teams only contain one thread)
- **`omp_get_ancestor_thread_num(`*`level`*`)`**
  - returns the thread ID of this thread's ancestor at a given level
  - ID of my parent:
    `omp_get_ancestor_thread_num(omp_get_level()-1)`
- **`omp_get_team_size(`*`level`*`)`**
  - returns the number of threads in this thread's ancestor team at a given level

Synchronisation in nested parallelism

- Note that barriers (explicit or implicit) only affect the innermost enclosing parallel region.
- No way to have a barrier across multiple teams

- In contrast, critical regions, atomics and locks affect all the threads in the program
- If you want mutual exclusion within teams but not between them, need to use locks (or atomics).

Nested loops

- For perfectly nested rectangular loops we can parallelise multiple loops in the nest with the `collapse` clause:

```
#pragma omp parallel for collapse(2)
for (int i=0; i<N; i++) {
  for (int j=0; j<M; j++) {
        .....
  }
}
```

- Argument is number of loops to collapse starting from the outside
- Will form a single loop of length NxM and then parallelise and schedule that.
- Useful if N is O(no. of threads) so parallelising the outer loop may not have good load balance
- More efficient than using nested teams

Orphaned directives

- Directives can be present in functions called from inside parallel regions
- Example:

```
#pragma omp parallel
{
    fred();
}

void fred() {
#pragma omp for
    for (int i=0; i<N; i++) {
        a[i] += 23.5;
    }
}
```

Orphaned directives (cont)

- This is very useful, as it allows a modular programming style….
- But it can also be rather confusing if the call tree is complicated (what happens if `fred` is also called from outside a parallel region? - the worksharing loop is all executed by the master thread)
- There are some extra rules about data scope attributes….

Data scoping rules

When we call a subroutine from inside a parallel region:

- Variables in the argument list passed by reference inherit their data scope attribute from the calling routine.
- Global variables are shared, unless declared `threadprivate` (see later).
- `static` local variables are shared.
- All other local variables are private.

Thread private global variables

- It can be convenient for each thread to have its own copy of variables with global scope
- Outside parallel regions and in **`masked`** directives, accesses to these variables refer to the inital thread's copy.

C/C++: **`#pragma omp threadprivate`** **(***list***)**

This directive must be at file or namespace scope, after all declarations of variables in *list* and before any references to variables in *list*. See standard document for other restrictions.

The **`copyin`** clause allows the values of the initial thread's **`threadprivate`** data to be copied to all other threads at the start of a parallel region.

User defined reductions

- Cannot use built-in reduction operators on objects or structures.

- Use `declare reduction` directive to define new reduction operators
- New operators can then be used in reduction clause.

```
#pragma omp declare reduction (reduction-identifier :
typename-list : combiner) [identity(identity-expr)]
```

- `reduction-identifier` gives a name to the operator
  - Can be overloaded for different types
  - Can be redefined in inner scopes
- `typename-list` is a list of types to which it applies
- `combiner` expression specifies how to combine values
- `identity` can specify the identity value of the operator

  Can be an expression or a brace initializer

Example

```
#pragma omp declare reduction (merge : std::vector<int>
: omp_out.insert(omp_out.end(), omp_in.begin(), omp_in.end()))
```

- Private copies created for a reduction are initialized to the identity that was specified for the operator and type
  - Default identity defined if identity clause not present
- Compiler uses combiner to combine private copies
- `omp_out` refers to private copy that holds combined values
- `omp_in` refers to the other private copy
- Can now use `merge` as a reduction operator.

Construct cancellation

- Clean way to signal early termination of an OpenMP construct.
  - one thread signals
  - other threads jump to the end of the construct

`#pragma omp cancel` *construct* **`[if (expr)]`**

cancels the construct where *construct* is **`parallel`**, **`sections`**, **`for`** or **`taskgroup`**

`#pragma omp cancellation point` *construct*

checks for cancellation (also happens implicitly at cancel directive, barriers etc.)

Example

```
#pragma omp parallel for private(eureka)
for (i=0; i<N; i++){
   eureka = testing(i,...);
#pragma omp cancel parallel if(eureka)
}
```

- First thread for which `eureka` is true will cancel the parallel region and exit.
- Other threads exit next time they hit the `cancel` directive

Timing routines

OpenMP supports a portable timer:

- return current wall clock time (relative to arbitrary origin) with:

```
double omp_get_wtime(void);
```

- no guarantee about clock precision but it can be queried with:

```
double omp_get_wtick(void);
```

Usage:

```
double starttime, time;
starttime = omp_get_wtime();
   ......(work to be timed)
time = omp_get_wtime()- starttime;
```

Timers are local to a thread: must make both calls on the same thread.

Exercise

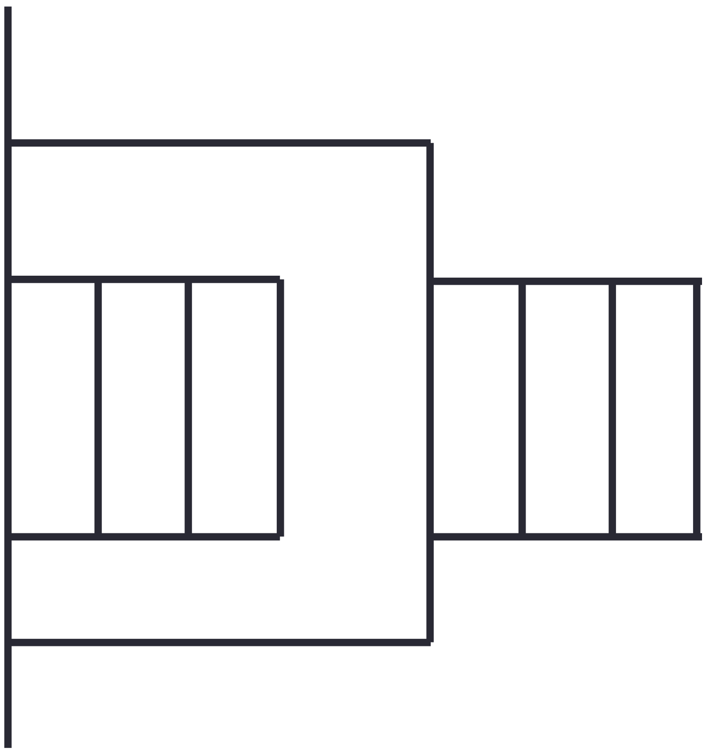
Molecular dynamics again

- Aim: use of orphaned directives.
- Modify the molecular dynamics code so by placing a parallel region directive around the iteration loop in the main program, and making *all* code within this sequential except for the forces loop.
- Don't expect this to change the performance much: the idea is to see how it affects data-attribute scoping
- Be careful about harmless-looking race conditions!

Fortran example - nested parallelism

Example:

```
!$OMP PARALLEL PRIVATE(myid)
myid = omp_get_thread_num()
if (myid .eq. 0) then
!$OMP PARALLEL DO
      do i = 1,n
         x(i) = 1.0
      end do
elseif (myid .eq.1) then
!$OMP PARALLEL DO
      do j = 1,n
         y(j) = 2.0
      end do
endif
!$OMP END PARALLEL
```



Fortran example - orphaned directives

```
!$OMP PARALLEL
      call fred()
!$OMP END PARALLEL

      subroutine fred()
!$OMP DO
      do i = 1,n
         a(i) = a(i) + 23.5
      end do
      return
      end
```

Fortran data scoping rules

When we call a subroutine/function from inside a parallel region:

- Variables in the argument list inherit their data scope attribute from the calling routine.
- COMMON blocks or module variables in Fortran are shared, unless declared **`THREADPRIVATE`** (see below).
- **`SAVE`** variables in Fortran are shared.
- All other local variables are private.

`!$OMP THREADPRIVATE (`*list*`)`

where list contains named common blocks (enclosed in slashes), module variables and **`SAVE`** variables..

- This directive must come after all the declarations for the COMMON blocks or variables.

Fortran example - using timers

```
DOUBLE PRECISION STARTTIME, TIME

STARTTIME = OMP_GET_WTIME()
......(work to be timed)
TIME = OMP_GET_WTIME()- STARTTIME
```

Reusing this material